UADE · TECNICATURA EN DESARROLLO DE SOFTWARE

TRABAJO PRÁCTICO OBLIGATORIO

PROGRAMACIÓN III

PLANIFICACIÓN DE RUTAS Y DISTRIBUCIÓN DE AYUDA

Propuesta 2 · Informe final

Integrantes: Ovelar Evangelina, Peralta Luka

Entrega final: 13 de noviembre de 2026

Repositorio: https://github.com/eovelar/TPO-PROGRAMACION-III

Índice

1. Introducción y alcance
2. Organización y consulta de pedidos
3. Selección de cargas prioritarias
4. Determinación de la ruta de menor tiempo
5. Integración de la primera etapa
6. A1 — Consultas de rutas entre todos los centros
7. B1 — Recorrido con múltiples entregas
8. B3 — Rutas alternativas con restricciones
9. Integración, pruebas y correcciones
10. Herramientas y conclusiones

1. Introducción y alcance

En este trabajo desarrollamos un sistema en Java para organizar la distribución de alimentos, medicamentos e insumos a distintos centros comunitarios. El programa trabaja con depósitos, centros, conexiones viales, vehículos, pedidos y lotes de mercadería. Después de la devolución de la primera entrega reorganizamos el proyecto en paquetes de modelo, algoritmos, servicios, persistencia, pruebas y vista. La versión final incluye las tres funcionalidades obligatorias (pedidos, Dijkstra y cargas) y las tres adicionales que elegimos: A1 con Floyd–Warshall, B1 para múltiples entregas y B3 para rutas alternativas. En cada sección explicamos cómo funciona la solución, las alternativas que consideramos, su complejidad y las pruebas.
Las seis funcionalidades están integradas en el menú de consola y el proyecto se trabaja en la rama main de GitHub. Además, agregamos una interfaz Swing para registrar y consultar pedidos, seleccionar cargas y visualizar rutas con Dijkstra y Floyd–Warshall. B1 y B3 se siguen ejecutando desde el menú de consola.

2. Organización y consulta de pedidos

2.1. Datos y requisitos

Cada pedido tiene un ID, un centro de destino, una fecha límite, una prioridad y un peso total. El sistema permite ordenar por fecha o por prioridad y buscar un pedido por su ID. Cuando dos pedidos tienen la misma prioridad, se compara primero la fecha límite y después el ID menor. El ordenamiento y la búsqueda fueron implementados sin usar métodos de Java que resuelvan directamente esas operaciones.

2.2. Implementación

La clase Pedido guarda los datos de cada pedido y valida que sean correctos al crearlo. GestorPedidos administra los pedidos en un ArrayList, evita IDs repetidos y llama a los algoritmos de ordenamiento y búsqueda. Para ordenar sin cambiar la lista original, primero convierte los pedidos en un arreglo nuevo.
Para ordenar los pedidos usamos MergeSortPedidos, que permite organizarlos por fecha, prioridad o ID. El método mergeSort divide el arreglo en mitades; comparar establece el orden y merge vuelve a unir las partes ya ordenadas. Para buscar un ID, GestorPedidos ordena una copia y utiliza BusquedaBinariaPedidos, que en cada paso descarta la mitad donde no puede encontrarse el pedido.

2.3. Estrategias, justificación y alternativas

Elegimos Merge Sort porque utiliza Divide y Conquista: divide el arreglo en partes más pequeñas y luego las vuelve a unir en orden. Su complejidad es O(n log n), incluso en el peor caso, por lo que resulta conveniente si aumenta la cantidad de pedidos. También consideramos Bubble Sort, pero su complejidad $O(n^2)$ es menos eficiente para muchos elementos.

La búsqueda binaria necesita que los pedidos estén ordenados por ID. Compara el elemento del medio con el ID buscado y continúa en una sola mitad del arreglo. Una búsqueda lineal sería más simple y no necesitaría ordenar los datos, pero puede revisar todos los pedidos. La búsqueda binaria sola tiene complejidad O(log n); como nuestro método ordena una copia antes de buscar, el costo total de buscarPedidoPorId es O(n log n).

2.4. Complejidad

Ordenamiento por fecha, prioridad o ID: tiempo O(n log n) y espacio auxiliar O(n).
Búsqueda binaria sobre arreglo ya ordenado: tiempo O(log n) y espacio O(1) en su implementación iterativa.
Búsqueda mediante GestorPedidos (copia + ordenamiento por ID + búsqueda binaria): tiempo O(n log n) y espacio O(n).
Agregar pedido con verificación de duplicado utiliza una búsqueda que vuelve a ordenar: tiempo O(n log n) para n pedidos existentes.

2.5. Pruebas realizadas

Para los pedidos con ID 103 (20/10, prioridad 5), 104 (17/10, prioridad 3), 102 (18/10, prioridad 5) y 101 (18/10, prioridad 5), se obtuvo el orden por fecha 104 → 101 → 102 → 103 y por prioridad 101 → 102 → 103 → 104. PruebaPedidos.java terminó con 19 pruebas correctas y 0 fallidas: verificamos fechas, prioridades, desempates, ID encontrado o ausente, primer y último elemento, arreglo vacío, orden inverso, pedido nulo, datos inválidos y duplicados.

Entrada	**Resultado esperado**	**Resultado obtenido**
IDs 103,104,102,101; ordenar por fecha	104 → 101 → 102 → 103	104 → 101 → 102 → 103
Mismos pedidos; ordenar por prioridad	101 → 102 → 103 → 104	101 → 102 → 103 → 104
Buscar ID 999	Pedido no encontrado	No encontrado

3. Selección de cargas prioritarias

3.1. Datos y objetivo

En esta funcionalidad trabajamos con vehículos de capacidad limitada y lotes que tienen un ID, un peso entero mayor que cero y una prioridad que puede ser cero o positiva. El objetivo es elegir la combinación con mayor prioridad total sin superar la

capacidad del vehículo. Los lotes no se pueden dividir ni cargar más de una vez. Este cálculo se hace por separado de las rutas.

3.2. Implementación

Las clases LoteMercaderia y Vehiculo guardan los datos de los lotes y del vehículo. También validan que los valores ingresados sean correctos. El algoritmo se encuentra en SeleccionCargas y muestra por consola los lotes elegidos, el peso utilizado, la prioridad total y la capacidad restante.
Para seleccionar las cargas usamos el algoritmo de Mochila 0/1 con Programación Dinámica. Construimos una tabla llamada dp: cada fila representa los lotes que llevamos analizados y cada columna, una capacidad posible. En cada casilla guardamos la mayor prioridad que podemos conseguir sin superar esa capacidad. Para cada lote comparamos dos opciones: dejarlo afuera o incluirlo si entra. La fórmula utilizada es dp[i][j] = max(dp[i−1][j], prioridad_i + dp[i−1][j−peso_i]). Al terminar, recorremos la tabla hacia atrás para identificar los lotes seleccionados. Si no hay lotes o la capacidad es cero, la selección queda vacía.

3.3. Estrategia y alternativas

Elegimos Programación Dinámica porque permite guardar resultados intermedios y reutilizarlos, evitando cálculos repetidos. Un método Greedy, como elegir primero el lote con mayor prioridad o con mejor relación entre prioridad y peso, no siempre encuentra la mejor combinación si los lotes no pueden dividirse. También podríamos usar Backtracking, pero tendría que explorar muchas más combinaciones en los casos más difíciles.

3.4. Complejidad

En este algoritmo, n representa la cantidad de lotes y C' es el menor valor entre la capacidad del vehículo y la suma de los pesos. El tiempo y la memoria utilizados son O(n · C'), porque construimos una tabla que considera distintas capacidades para cada lote. Es decir, el trabajo aumenta tanto con la cantidad de lotes como con la capacidad disponible.

3.5. Pruebas realizadas

Con un vehículo de capacidad 50 y lotes de datos (ID 1: peso 10, prioridad 30; ID 2: peso 20, prioridad 40; ID 3: peso 30, prioridad 50; ID 4: peso 40, prioridad 60), el algoritmo seleccionó los lotes 2 y 3. Se obtuvo un peso total de 50, prioridad acumulada de 90 y capacidad restante igual a cero.
En la versión final, PruebaCargas.java terminó con 10 pruebas correctas y 0 fallidas. Comprobamos lista vacía, capacidad cero, capacidad mayor que la suma de pesos, ningún lote que entre, empate de prioridad, entradas negativas o nulas y un caso en que una elección Greedy no da el máximo. Para capacidad 50 y lotes (10,60), (20,100) y (30,120), el resultado fue seleccionar los últimos dos: peso 50 y prioridad 220.

Entrada	Resultado esperado	Resultado obtenido
Capacidad 50; lotes (10,30), (20,40), (30,50), (40,60)	Lotes 2 y 3; prioridad 90	Lotes 2 y 3; prioridad 90
Capacidad 0; lotes disponibles	Sin lotes; prioridad 0	Sin lotes; prioridad 0
Capacidad 50; lotes (10,60), (20,100), (30,120)	Lotes 2 y 3; prioridad 220	Lotes 2 y 3; prioridad 220

4. Determinación de la ruta de menor tiempo

4.1. Datos y restricciones

La red vial está formada por depósitos, centros comunitarios y conexiones con un sentido de circulación y un tiempo positivo en minutos. Algunas conexiones pueden estar deshabilitadas. Al indicar un origen y un destino, el programa busca el recorrido de menor tiempo y muestra las ubicaciones por las que pasa. Si se deshabilita una conexión, se puede volver a calcular la ruta, y si no existe un camino posible, el programa lo informa.

4.2. Estructura del módulo

Para trabajar con rutas usamos las clases ConexionVial, RedVial, Dijkstra, ResultadoRuta y PruebaRutas. ConexionVial representa cada conexión y su estado; RedVial guarda las ubicaciones y conexiones; Dijkstra calcula el camino más rápido; ResultadoRuta guarda el recorrido, el tiempo y las decisiones tomadas; y PruebaRutas permite comprobar distintos casos.

4.3. Selección y fundamento de la estrategia

Elegimos Dijkstra porque permite obtener el camino más corto cuando los tiempos de las conexiones son positivos. Aplica la estrategia Greedy: en cada paso selecciona la ubicación no visitada con menor tiempo acumulado y revisa si puede mejorar las distancias hacia sus vecinos. También guarda desde qué ubicación se llegó a cada una, para después reconstruir el recorrido.
Una alternativa sería recorrer todos los caminos posibles y quedarse con el más corto, pero eso puede requerir muchas combinaciones. También consideramos Floyd–Warshall, que sirve para obtener caminos mínimos entre todos los pares de ubicaciones. Para esta primera etapa elegimos Dijkstra porque las consultas parten de un origen y un destino concretos; Floyd–Warshall queda como opción para la funcionalidad A1.

4.4. Complejidad

En nuestra implementación, Dijkstra busca la ubicación con menor tiempo acumulado revisando las que todavía no procesó. Además, RedVial recorre las conexiones para encontrar las que salen de cada lugar. Si V es la cantidad de ubicaciones y E la cantidad de conexiones, el tiempo de ejecución es, como máximo, $O(V^2 + V \cdot E)$. No usamos una cola de prioridad. Para guardar las distancias, los puntos anteriores y las ubicaciones pendientes se necesita O(V) de memoria adicional, sin contar los registros de decisiones ni las listas temporales.

4.5. Casos de prueba y resultados obtenidos

Caso 1 — Ruta de menor tiempo: desde Depósito Central hasta Centro Este se obtuvo el recorrido Depósito Central → Centro Norte → Centro Este, con un tiempo total de 15 minutos. Resultado satisfactorio.
Caso 2 — Modificación de la red: al deshabilitar la conexión Centro Norte → Centro Este, el sistema recalculó el recorrido Depósito Central → Centro Sur → Centro Este, con una duración de 23 minutos. Resultado satisfactorio.
Caso 3 — Destino inaccesible: en la versión final usamos una red de prueba con dos ubicaciones sin conexión. Dijkstra informó correctamente que no existía camino. No usamos Centro Este → Depósito Central como caso inaccesible porque agregamos conexiones de regreso para el recorrido de múltiples entregas.
Caso 4 — Origen y destino coincidentes: el algoritmo devolvió el Depósito Central como único punto del recorrido y un tiempo total de 0 minutos. Resultado satisfactorio.
También verificamos que las conexiones tienen un sentido de circulación: poder ir de un lugar a otro no significa que exista automáticamente una conexión para regresar. Durante las pruebas también registramos las ubicaciones visitadas, los cambios en los tiempos calculados y si se alcanzó el destino. Esto nos permite revisar las decisiones que tomó el algoritmo.
En la versión final, PruebaRutas.java terminó con 10 pruebas correctas y 0 fallidas. Además del recorrido 1 → 2 → 4 de 15 minutos y el recálculo de 23 minutos al deshabilitar 2 → 4, se probaron el origen igual al destino, un destino aislado, valores nulos, ubicaciones ajenas a la red y la restauración de conexiones.

Entrada	**Resultado esperado**	**Resultado obtenido**
Origen 1, destino 4	1 → 2 → 4; 15 min	1 → 2 → 4; 15 min
Deshabilitar 2 → 4; consultar 1 → 4	1 → 3 → 4; 23 min	1 → 3 → 4; 23 min
Dos ubicaciones sin conexión	Sin recorrido	Sin recorrido

5. Integración de la primera etapa

5.1. Interfaz de consola y organización

Main.java permite ejecutar las seis funcionalidades desde consola. Como complemento, desarrollamos NEXO en Java Swing: PanelPedidos registra, ordena y busca pedidos; PanelCargas permite ingresar lotes y ejecutar la mochila 0/1; PanelRutas consulta Dijkstra y Floyd–Warshall y PanelGrafo dibuja la red con el recorrido destacado. B1 y B3 permanecen disponibles desde la consola. Los paneles reutilizan los algoritmos existentes, sin volver a implementarlos.

5.2. Pruebas de los módulos ya ejecutadas

En las pruebas finales de las funcionalidades obligatorias obtuvimos 19/19 en pedidos, 10/10 en cargas y 10/10 en Dijkstra. El conjunto general TestAlgoritmos.java volvió a terminar con 10 correctas y 0 fallidas. Las baterías adicionales y el total se resumen en la sección 9.

5.3. Integración de los módulos

Integramos los módulos originales de ambos integrantes en la rama main. La ejecución desde Main permitió agregar un pedido, ordenar por fecha, consultar Dijkstra y Floyd–Warshall, planificar tres entregas, buscar rutas con prohibiciones y seleccionar cargas. En todos los ejemplos ejecutados se obtuvieron los resultados esperados. Como mejora posterior incorporamos controles para evitar pedir más centros comunitarios de los disponibles y permitir cancelar la selección. De este modo el menú no queda atrapado cuando se ingresa una cantidad imposible.

6. A1 — Consultas de rutas entre todos los centros

6.1. Problema y datos

En esta funcionalidad necesitamos conocer las rutas entre distintas ubicaciones de la red, sin tener que hacer un cálculo nuevo para cada consulta. La red conserva las mismas conexiones dirigidas y sus tiempos positivos. También tiene que indicar cuándo dos lugares no están conectados y permitir reconstruir las ubicaciones del recorrido.

6.2. Estrategia e implementación

Usamos Floyd–Warshall, un algoritmo de Programación Dinámica. La clase FloydWarshall.java utiliza dos matrices: una para los tiempos mínimos y otra para reconstruir los recorridos. Al iniciar el cálculo, cada ubicación tiene tiempo cero hacia sí misma, y las ubicaciones sin conexión directa se consideran inaccesibles. Después probamos si pasar por una ubicación intermedia permite llegar más rápido. Cuando

encontramos un recorrido mejor, actualizamos el tiempo y guardamos el siguiente paso.
La consulta se hace con consultar(idOrigen, idDestino). Una vez construidas las matrices, podemos consultar diferentes pares sin repetir el cálculo. Cuando se habilita o deshabilita una conexión, Main llama nuevamente a calcular(), porque los valores precalculados ya podrían haber cambiado.

6.3. Justificación y alternativas

Elegimos Floyd–Warshall porque A1 necesita consultar rutas entre muchos pares de ubicaciones. Con Dijkstra también podríamos hacerlo, pero tendríamos que repetir el cálculo desde distintos orígenes. Floyd–Warshall requiere más trabajo al principio, aunque después permite consultar los resultados que quedaron guardados en las matrices.

6.4. Complejidad

Floyd–Warshall utiliza tres ciclos que recorren las V ubicaciones de la red, por eso su complejidad temporal es $O(V^3)$. La carga inicial de conexiones agrega $O(E \cdot V)$, donde E es la cantidad de conexiones. Las matrices necesitan $O(V^2)$ de memoria. Una vez realizado el cálculo, cada consulta requiere $O(V + L)$: V para localizar los identificadores y L para reconstruir las ubicaciones del camino.

6.5. Pruebas

PruebaFloydWarshall.java terminó con 10 pruebas correctas y 0 fallidas. Con la red de ejemplo, de Depósito Central (1) a Centro Este (4) se obtuvo 1 → 2 → 4 en 15 minutos. Una segunda consulta 1 → 3 dio 15 minutos, sin volver a calcular. También comprobamos 1 → 1 en 0 minutos y que desde Depósito Secundario (5) hasta Centro Este (4) no hay recorrido.
Al deshabilitar la conexión 2 → 4 y recalcular, el resultado 1 → 4 pasó a 1 → 3 → 4, con 23 minutos; al restaurarla volvió a 15. Se probaron además una ubicación inexistente, una consulta antes del cálculo y consultas repetidas. Los resultados coincidieron con los esperados.

Entrada	**Resultado esperado**	**Resultado obtenido**
Consulta 1 → 4, matrices calculadas	1 → 2 → 4; 15 min	1 → 2 → 4; 15 min
Deshabilitar 2 → 4 y recalcular	1 → 3 → 4; 23 min	1 → 3 → 4; 23 min
Consulta 5 → 4	Sin recorrido	Sin recorrido

7. B1 — Recorrido con múltiples entregas

7.1. Problema y restricciones

Un vehículo debe salir de un depósito, visitar los centros comunitarios seleccionados y regresar al depósito, procurando el menor tiempo total. Se permite elegir de uno a ocho centros y cada uno debe contarse una sola vez como parada de entrega. El camino más corto entre dos paradas puede atravesar otras ubicaciones de la red; eso no significa que se esté realizando allí una entrega.

7.2. Estrategia e implementación

Implementamos Backtracking en RecorridoMultiplesEntregas.java. Con un arreglo de visitados y una lista de ordenActual probamos distintos órdenes para los centros elegidos. En cada avance consultamos a FloydWarshall el tiempo mínimo entre la parada actual y la siguiente. Cuando ya se visitaron todos, sumamos el regreso al depósito y comparamos con menorTiempo. El mejor orden se guarda en mejorOrden y después se imprime también el recorrido completo de las conexiones utilizadas.
Si no existe un camino entre dos paradas, descartamos esa posibilidad. También usamos una poda: cuando el tiempo acumulado ya es igual o mayor al mejor que encontramos, dejamos de explorar ese orden. Esto es correcto porque los tramos tienen tiempos positivos y agregar más paradas no puede reducir el total. Después retrocedemos, desmarcamos el último centro y probamos otro orden.

7.3. Justificación y alternativas

Elegimos Backtracking porque necesitamos comparar distintos órdenes de entrega y encontrar el más rápido. Ir siempre al centro más cercano, como haría una estrategia Greedy, no garantiza el mejor recorrido completo: también importan las siguientes entregas y el regreso al depósito. Podríamos probar todos los órdenes sin descartar ninguno, pero eso implicaría revisar opciones que ya sabemos que no van a mejorar el resultado.

7.4. Complejidad

Si elegimos k centros, existen hasta k! órdenes posibles de visita. Por eso, la búsqueda tiene una complejidad de $O(k! \cdot k)$ en el peor caso, además del cálculo previo de Floyd–Warshall, que cuesta $O(V^3)$. Las podas ayudan a evitar recorridos innecesarios, pero no eliminan el peor caso factorial. Para guardar el orden parcial y las visitas usamos $O(k)$ de memoria adicional, sin contar las matrices de rutas.

7.5. Pruebas

PruebaRecorridoMultiples.java terminó con 16 pruebas correctas y 0 fallidas. Con depósito 1 y centros 2, 3 y 4, encontramos el orden Depósito Central → Centro Norte

→ Centro Sur → Centro Este → Depósito Central, con 56 minutos. Comparamos el tiempo obtenido con una búsqueda exhaustiva de órdenes sin podas y coincidieron. También probamos uno y dos centros, reinicio de resultados, depósitos sin salida, regreso bloqueado, restauración de una conexión y entradas inválidas (centros repetidos, lista vacía, depósito incorrecto y más de ocho centros). Aún es conveniente sumar una prueba con exactamente ocho centros para cubrir el límite superior. En el menú Main corregimos además un problema de interacción: con solo tres centros cargados ya no se pueden solicitar siete entregas; el programa informa el máximo disponible y permite cancelar con 0.

Entrada	Resultado esperado	Resultado obtenido
Depósito 1; centros 2, 3 y 4	Circuito óptimo; 56 min	1 → 2 → 3 → 4 → 1; 56 min
Depósito 1; solo centro 2	1 → 2 → 1; 20 min	1 → 2 → 1; 20 min
Depósito 5; entrega al centro 2	Sin circuito completo	Sin circuito completo

8. B3 — Rutas alternativas con restricciones

8.1. Problema y restricciones

Se deben encontrar todos los recorridos válidos desde un origen hasta un destino, sin superar un tiempo máximo, sin atravesar ubicaciones prohibidas y sin repetir ubicaciones en un mismo camino. Las conexiones son dirigidas y una conexión deshabilitada no puede utilizarse. A diferencia de Dijkstra, aquí no basta con devolver la ruta más corta.

8.2. Estrategia e implementación

Usamos Backtracking en RutasAlternativas.java. El método buscar() recibe los IDs de origen y destino, el límite de tiempo y un conjunto de ubicaciones prohibidas. El método recursivo recorre las conexiones habilitadas, acumula minutos y mantiene las ubicaciones visitadas. Si llega al destino, guarda una copia del recorrido y su tiempo. Después retrocede para seguir buscando otras rutas.
Dejamos de explorar un recorrido cuando pasa por un lugar prohibido, repite una ubicación o supera el tiempo máximo. Como esas opciones ya incumplen las reglas, descartarlas no hace que perdamos rutas válidas. Si no encontramos ninguna, el programa lo informa.

8.3. Justificación y alternativas

Backtracking permite enumerar todas las soluciones válidas. Dijkstra serviría para hallar la más rápida, pero no para mostrar las demás. También se podría generar todos

los caminos y filtrarlos al final, aunque sería menos eficiente porque seguiría explorando recorridos que ya incumplieron las restricciones.

8.4. Complejidad

La cantidad de rutas posibles sin repetir ubicaciones puede crecer muy rápido cuando aumenta el tamaño de la red. En el peor caso, la búsqueda puede necesitar un tiempo factorial respecto de V, la cantidad de ubicaciones. Las restricciones de tiempo y los lugares prohibidos ayudan a descartar opciones, aunque no cambian ese peor caso. Para las llamadas recursivas y el recorrido actual usamos O(V) de memoria, además del espacio necesario para guardar todas las rutas encontradas.

8.5. Pruebas

PruebaRutasAlternativas.java terminó con 16 pruebas correctas y 0 fallidas. Para origen 1, destino 4 y máximo de 30 minutos se encontraron dos caminos: 1 → 2 → 4 (15 minutos) y 1 → 3 → 4 (23 minutos). Con máximo de 40 minutos aparece además 1 → 2 → 3 → 4 (38 minutos).
Al prohibir Centro Norte (2), solo quedó el recorrido 1 → 3 → 4 de 23 minutos. Con un límite de 10 minutos no apareció ninguna ruta válida. También se probaron el tiempo justo de 15 minutos, origen o destino prohibido, conexiones deshabilitadas, origen igual a destino, ubicaciones inexistentes, tiempo negativo y limpieza de resultados entre ejecuciones.

Entrada	**Resultado esperado**	**Resultado obtenido**
Origen 1, destino 4; máximo 30 min	2 rutas: 15 y 23 min	2 rutas: 15 y 23 min
Prohibido centro 2; máximo 30 min	1 → 3 → 4; 23 min	1 → 3 → 4; 23 min
Origen 1, destino 4; máximo 10 min	Sin rutas válidas	Sin rutas válidas

9. Integración, pruebas y correcciones

9.1. Organización actual del proyecto

Después de la primera devolución reorganizamos el proyecto por responsabilidades. El paquete modelo contiene los datos del problema; algoritmos reúne MergeSortPedidos, BusquedaBinariaPedidos, SeleccionCargas, Dijkstra, FloydWarshall y los dos Backtracking; servicios contiene GestorPedidos; rutas conserva RedVial y ResultadoRuta; persistencia incluye LectorDatos; pruebas reúne las clases de

verificación; y vista contiene Main, VentanaPrincipal, PanelPedidos, PanelCargas, PanelRutas y PanelGrafo.
La red vial se carga desde datos/ubicaciones.txt y datos/conexiones.txt, de modo que podemos cambiar los lugares y tiempos sin recompilar. Las operaciones de pedidos y cargas también admiten datos ingresados por menú. Los algoritmos permanecen separados de la presentación, lo que permite ejecutarlos y probarlos sin abrir la interfaz gráfica.

La estructura principal del proyecto quedó organizada de la siguiente manera:

```
TPO-PROGRAMACION-III/
├── src/
│   └── distribucion/
│       ├── modelo/
│       ├── algoritmos/
│       ├── servicios/
│       ├── rutas/
│       ├── persistencia/
│       ├── pruebas/
│       └── vista/
├── datos/
│   ├── ubicaciones.txt
│   └── conexiones.txt
└── README.md
```

Esta separación facilita encontrar las clases y modificar la interfaz sin tener que cambiar los algoritmos.

9.2. Ejecución y resultados

La consola Main integra las seis funcionalidades. En una ejecución manual agregamos un pedido, consultamos los ordenamientos, calculamos rutas con Dijkstra y Floyd–Warshall, ejecutamos B1 con tres centros y B3 con Centro Norte prohibido. La selección de cargas también se probó desde el menú con vehículo de 50 kg y lotes (10,60), (20,100) y (30,120): seleccionó los dos últimos, con prioridad 220 y peso 50 kg.
Las pruebas terminaron con 19/19 casos correctos en pedidos, 10/10 en cargas, 10/10 en Dijkstra, 10/10 en Floyd–Warshall, 16/16 en B1, 16/16 en B3 y 10/10 en TestAlgoritmos. En total fueron 91 pruebas aprobadas y ninguna fallida. Aunque siempre podrían existir situaciones no contempladas, verificamos los ejemplos principales y varios casos especiales.

9.3. Correcciones de la primera entrega

El profesor valoró positivamente las tres funcionalidades obligatorias y el informe parcial. Nos observó que la organización por funcionalidad era poco escalable y que convenía contar con datos de prueba más flexibles. Reorganizamos los paquetes en modelo, algoritmos, servicios y vista, agregamos persistencia para leer TXT y ampliamos las pruebas automatizadas. También corregimos el manejo de capacidad cero en Vehiculo y SeleccionCargas y la selección de centros de B1 desde el menú.

9.4. Estrategias utilizadas

Divide y Conquista: Merge Sort de pedidos. Greedy: Dijkstra para el camino de menor tiempo. Programación Dinámica: Mochila 0/1 y Floyd–Warshall. Backtracking: recorrido con múltiples entregas y enumeración de rutas alternativas. La búsqueda binaria complementa el ordenamiento de pedidos.

10. Herramientas y conclusiones

10.1. Herramientas utilizadas

Para desarrollar el proyecto utilizamos Java, IntelliJ IDEA, Git y GitHub. También utilizamos herramientas de inteligencia artificial como apoyo para resolver dudas, revisar código y preparar algunos casos de prueba. El código y los resultados presentados se comprobaron mediante las pruebas realizadas en el sistema.

10.2. Conclusiones

El trabajo nos permitió aplicar distintas formas de resolver problemas: ordenar pedidos, buscar caminos, elegir cargas y explorar recorridos con restricciones. Fue útil ver que no hay una técnica que sirva igual para todo; en algunos casos conviene dividir el problema, en otros tomar una decisión voraz, guardar resultados parciales o explorar opciones y retroceder.
Una de las dificultades fue reunir las distintas partes del proyecto y organizar las clases para que el código fuera más fácil de mantener. También aparecieron casos que no habíamos contemplado, como la capacidad cero o pedir más centros de los disponibles. Las pruebas nos ayudaron a encontrarlos y corregirlos. La interfaz gráfica se fue ampliando para incorporar pedidos y cargas además de las rutas. Como mejora futura queda sumar B1 y B3 a Swing y ampliar los datos de prueba, aunque todas las funcionalidades ya pueden ejecutarse desde la consola.

Utilizamos GitHub para mantener el código del proyecto y reunir las funcionalidades y correcciones realizadas durante el desarrollo.